Scheduling: Proportional Share

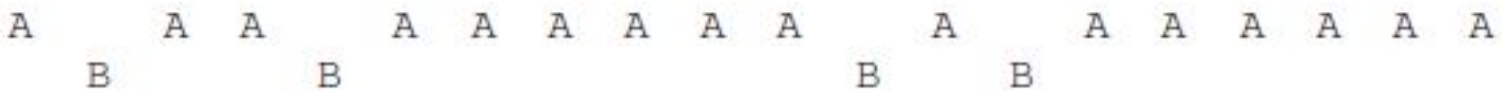
Лекция №6

Basic Concept: Tickets Represent Your Share

example output of a lottery scheduler's winning tickets

63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49 49

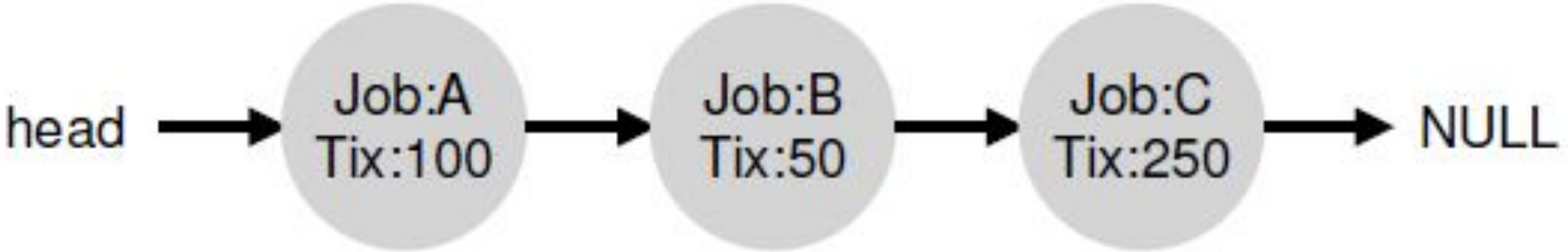
the resulting schedule

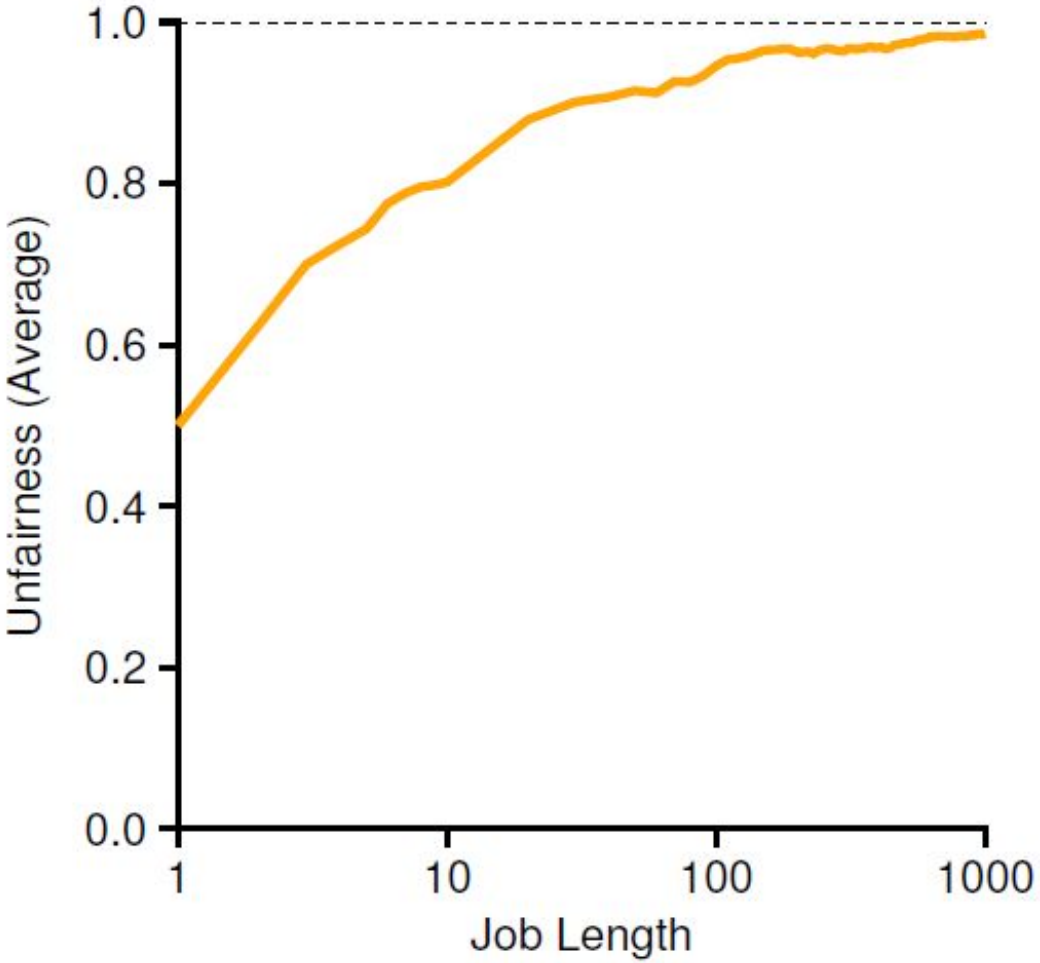


Ticket Mechanisms

```
1   // counter: used to track if we've found the winner yet
2   int counter = 0;
3
4   // winner: use some call to a random number generator to
5   //         get a value, between 0 and the total # of tickets
6   int winner = getrandom(0, totaltickets);
7
8   // current: use this to walk through the list of jobs
9   node_t *current = head;
10
11  // loop until the sum of ticket values is > the winner
12  while (current) {
13      counter = counter + current->tickets;
14      if (counter > winner)
15          break; // found the winner
16      current = current->next;
17  }
18  // 'current' is the winner: schedule it...
```

Lottery Scheduling Decision Code


head
Job:A
Tix:100
Job:B
Tix:50
Job:C
Tix:250
NULL



Lottery Fairness Study

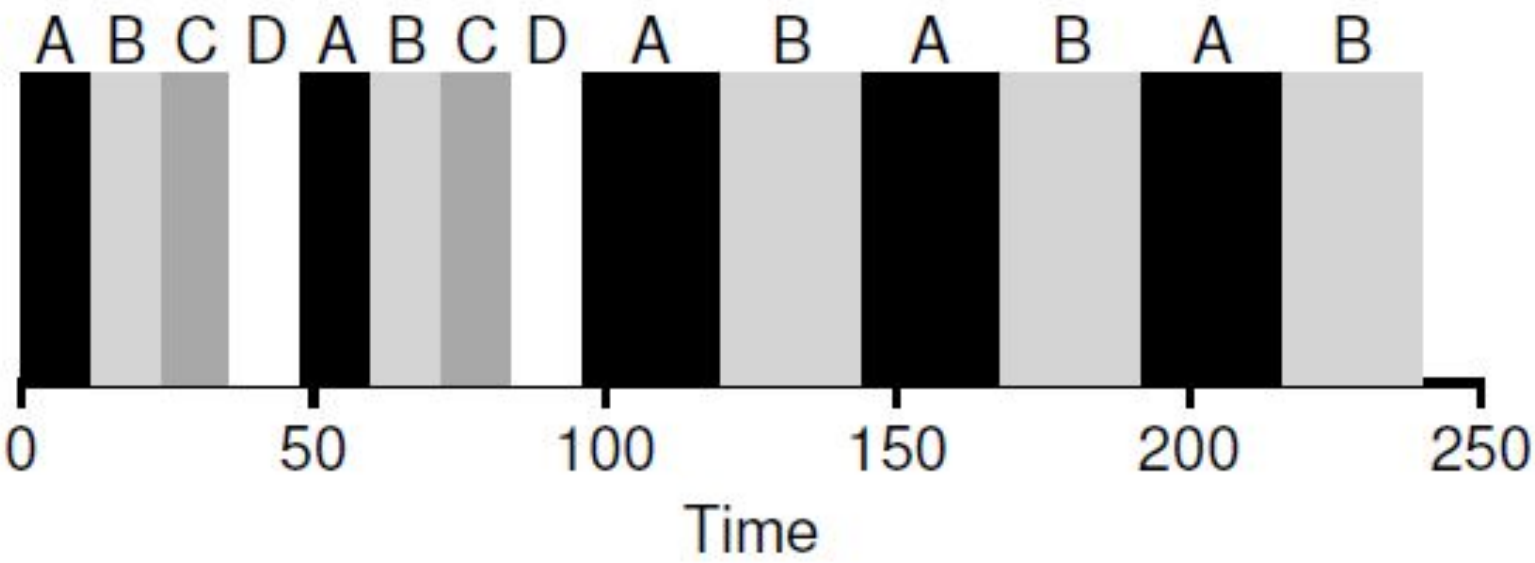
Псевдокод алгоритма со stride, pass value

```
current = remove_min(queue);       // pick client with minimum pass
schedule(current);                 // use resource for quantum
current->pass += current->stride; // compute next pass using stride
insert(queue, current);            // put back into the queue
```

Stride Scheduling: A Trace

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

The Linux Completely Fair Scheduler (CFS)



CFS Simple Example

CFS Red-Black Tree

